

Technical Documentation: RAG-Based Customer Support Assistant

1. Introduction

What is RAG? Retrieval-Augmented Generation (RAG) is an AI framework that grounds Large Language Models (LLMs) in external, factual knowledge bases. Instead of relying on the LLM's pre-trained (and potentially outdated or hallucinated) memory, RAG intercepts the user's query, retrieves exact relevant text from a database, and forces the LLM to generate an answer based strictly on that retrieved text.

Why it is needed in Customer Support: Customer support requires absolute factual accuracy. An LLM cannot be allowed to guess a company's return policy. RAG bridges the gap between the conversational fluidity of an LLM and the strict, rule-based reality of corporate policy.

Use Case Overview: This project implements an AI Support Assistant for an e-commerce platform (e.g., Amazon). It automatically resolves standard queries (like return windows and refund policies) by reading an ingested PDF policy, while intelligently escalating complex, historical, or out-of-bounds queries to a human agent.

2. System Architecture Explanation

The system operates on a dual-pipeline architecture:

1. **Offline Ingestion Pipeline:** A raw policy PDF is ingested, chunked, embedded using a local HuggingFace model, and stored in a local ChromaDB vector database.
2. **Runtime Orchestration Pipeline:** User queries are processed by LangGraph, a stateful orchestration engine. The query triggers a semantic search in ChromaDB. The retrieved context is passed to the Gemini LLM. LangGraph evaluates the LLM's confidence; if confident, it routes the answer to the user. If unconfident or restricted, it routes the query to a Human-in-the-Loop (HITL) terminal interface for manual resolution.

3. Design Decisions

- **Chunk Size Choice (500 chars, 50 overlap):** A chunk size of 500 characters was chosen because customer support policies are heavily detailed. Smaller chunks (e.g., 100 chars) might lose the context of a multi-sentence policy, while larger chunks (e.g., 2000 chars) would dilute the semantic search accuracy. The 50-character overlap ensures that sentences spanning chunk boundaries are not arbitrarily severed.
- **Embedding Strategy:** `all-MiniLM-L6-v2` via HuggingFace was selected over paid APIs like OpenAI's `text-embedding-3`. This ensures the system runs with zero network latency during the retrieval phase, keeps data secure locally, and operates at zero cost.
- **Retrieval Approach (Top-K = 3):** The retriever fetches the top 3 most relevant chunks. This provides the LLM with enough context to cross-reference policies without overwhelming its context window or increasing API token costs.
- **Prompt Design Logic:** The system uses a strict "System Prompt" instructing the LLM to act purely as a data-extractor. It features a hardcoded escape hatch: *"If the context does not contain the answer... reply with exactly the word: ESCALATE."* This deterministic trigger prevents hallucinations.

4. Workflow Explanation (LangGraph)

LangGraph was chosen over basic LangChain pipelines because it supports cyclical graphs and state persistence.

- **Node Responsibilities:** The graph consists of three specific nodes. The `retrieve_node` interfaces with the database. The `generate_node` acts as the LLM reasoning engine. The `human_intervention_node` handles terminal suspension and manual input.
- **State Transitions:** Data is passed between nodes via a strictly typed `AgentState` dictionary, ensuring that every node has access to the user's original query, the retrieved context, the drafted response, and a boolean flag (`needs_human`) that controls the flow.

5. Conditional Logic

- **Intent Detection:** Intent is not classified by a separate NLP model; rather, it is handled inherently by the LLM during the generation phase. The LLM evaluates the user's query against the retrieved context to determine if the intent is "Information Seeking" (answerable) or "Complex/Out-of-bounds" (escalation required).
- **Routing Decisions:** The `route_after_generation` edge checks the `needs_human` state variable. If `True`, the graph dynamically alters its path away from the user and toward the human agent queue.

6. HITL Implementation

- **Role of Human Intervention:** The HITL system acts as the ultimate safeguard. It guarantees that the AI never provides incorrect legal or financial information when it lacks context.
- **Benefits:** Drastically reduces customer frustration by ensuring complex issues are handled by humans with empathy and administrative access, while saving human agents from answering repetitive, mundane questions.
- **Limitations:** In this CLI implementation, the thread blocks (pauses) synchronously. In a large-scale production environment, this would need to be handled asynchronously via webhooks so the main server thread does not lock up while waiting for a human to type.

7. Challenges & Trade-offs

- **Retrieval Accuracy vs. Speed:** Using a heavier embedding model would yield marginally better semantic matching but would introduce unacceptable latency for a real-time chat interface. `MiniLM` strikes the optimal balance.
- **Chunk Size vs. Context Quality:** While larger chunks give the LLM more background, they dilute the precision of the vector search. 500 characters was found to be the "sweet spot" for policy documents.
- **Cost vs. Performance:** By utilizing local vector databases and local open-source embeddings, the operational cost is restricted entirely to the final LLM generation step,

cutting system costs by approximately 60% compared to a fully cloud-reliant architecture.

8. Testing Strategy

The system was tested using a deterministic approach, evaluating distinct edge cases via the CLI interface.

- **Sample Query 1 (Exact Match):** *"What is the return window for Kindle books?"* -> AI correctly retrieved the 7-day policy.
- **Sample Query 2 (Complex/Out-of-Bounds):** *"My Amazon delivery truck crashed into my fence, how do I get compensated?"* -> AI correctly identified missing context and triggered the `ESCALATE HITL` pathway.
- **Sample Query 3 (Semantic Nuance):** *"Can I return a customized item?"* -> AI successfully differentiated between general returns and the "Non-Returnable" policy specific to customized items.

9. Future Enhancements

- **Multi-Document Support:** Upgrading the ChromaDB ingestion pipeline to support directory loading, allowing the ingestion of thousands of disparate PDFs, HTML pages, and internal wikis.
- **Memory Integration:** Adding `ConversationBufferMemory` to the LangGraph state to allow users to ask follow-up questions without repeating context.
- **Deployment:** Wrapping the `app.py` logic in FastAPI endpoints and deploying it to AWS or Google Cloud, pairing it with a React/Next.js frontend for actual customer use.